

POSTMAN COLLECTION TO PYTEST MIGRATOR

Project Documentation

Submitted By

Mohamed Ashiq T – Computer Science and Engineering (CSE)

Harini R – Electronics and Communication Engineering (ECE)

Dhiviya K – Electronics and Communication Engineering (ECE)

Nithish S – Computer Science and Engineering (CSE)

Institution

J.J. College of Engineering and Technology

1. ABSTRACT

Postman Collection to Pytest Migrator is an AI-assisted automation platform designed to simplify the migration of API test collections from Postman into executable Pytest automation scripts. Organizations widely use Postman for API development and testing, while Pytest is one of the most popular frameworks for Python-based automation testing. However, converting Postman assertions and test logic into Pytest scripts manually is a repetitive and time-consuming task.

The proposed system automates this migration process by parsing Postman Collection JSON files, extracting API requests, headers, payloads, variables, and assertions, and converting them into executable Pytest scripts. A Hybrid Conversion Engine is implemented that combines a Rule-Based Engine and Google Gemini AI to handle both simple and complex assertions efficiently.

The system also generates recommendations, test cases, validation reports, and downloadable automation packages. User authentication, session management, and SQLite-based data storage ensure secure and organized operation.

The platform significantly reduces manual effort, improves migration accuracy, accelerates automation adoption, and enhances productivity for QA engineers and software development teams.

2. INTRODUCTION

API testing plays a crucial role in modern software development. Postman is one of the most widely used tools for API testing because of its user-friendly interface and powerful assertion capabilities.

As organizations scale their testing processes, they often adopt automation frameworks such as Pytest. Migrating existing Postman collections into Pytest scripts requires manual rewriting of assertions and request logic, which can be labor-intensive and error-prone.

The Postman Collection to Pytest Migrator addresses this challenge by providing an automated platform that converts Postman collections into executable Pytest scripts while generating recommendations and reports for testers.

3. PROBLEM STATEMENT

QA engineers frequently maintain hundreds of API test cases inside Postman collections. When migrating to automation frameworks such as Pytest, these tests must be rewritten manually.

Challenges

- Time-consuming migration process
- High possibility of human errors
- Difficult maintenance of large collections
- Repetitive and monotonous work
- Increased development effort

Manual Migration	Automated Migration
High effort	Low effort
Error-prone	Accurate
Slow process	Faster execution
Difficult maintenance	Easy management

4. OBJECTIVES

The objectives of the project are:

1. Automate Postman-to-Pytest migration.
2. Reduce manual conversion effort.
3. Generate executable Pytest scripts.

- 4. Validate uploaded collections.
 - 5. Generate recommendations and test cases.
 - 6. Improve testing productivity.
 - 7. Provide downloadable reports and scripts.
 - 8. Maintain secure user-specific data.
-

5. PROPOSED SOLUTION

The platform provides an automated workflow for converting Postman collections into Pytest automation scripts.

Workflow

User Login ↓ Upload Collection ↓ Validation Engine ↓ Parser Engine ↓ Hybrid Conversion Engine ↓ Pytest Generator ↓ Recommendation Engine ↓ Report Generator ↓ Download Center

Benefits

- Faster migration
 - Improved accuracy
 - Reduced manual work
 - Automated recommendation generation
 - Secure multi-user environment
-

6. TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User Interface
Backend	Flask, Python	Application Logic
Database	SQLite	Data Storage
AI Layer	Google Gemini API	Complex Assertion Conversion
Testing	Pytest	Automation Framework
Deployment	Render	Cloud Hosting

7. SYSTEM ARCHITECTURE

Architecture Flow

User ↓ Authentication Module ↓ Dashboard ↓ Collection Upload ↓ Parser Engine ↓ Validation Engine ↓ Hybrid Conversion Engine ↓ Pytest Generator ↓ Recommendations ↓ Reports ↓ Download Center

Major Modules

Authentication Module

Handles user registration, login, logout, and session management.

Upload Module

Accepts Postman Collection JSON files and stores metadata.

Parser Module

Extracts API details, request information, and assertions.

Validator Module

Validates collection structure and detects errors.

Hybrid Conversion Engine

Converts Postman assertions into Pytest assertions using rule-based logic and AI.

Report Generator

Creates reports, recommendations, and downloadable outputs.

8. DATABASE DESIGN

Database Used

SQLite

Database Relationship

Users | └─ Uploaded Collections | | └─ API Details | └─ Generated Scripts | └─ Generated Test Cases | └─ AI Recommendations



Users Table

Column	Type	Description
id	INTEGER	Primary Key
full_name	TEXT	User Name
email	TEXT	User Email
password_hash	TEXT	Encrypted Password
created_at	DATETIME	Registration Time

Purpose

Stores registered user information securely.

Uploaded_Collections Table

Column	Type	Description
id	INTEGER	Primary Key
user_id	INTEGER	Owner User
collection_name	TEXT	Collection Name
uploaded_at	DATETIME	Upload Time

Purpose

Stores uploaded Postman collections.

API_Details Table

Column	Type	Description
id	INTEGER	Primary Key
collection_id	INTEGER	Parent Collection
api_name	TEXT	API Name
method	TEXT	HTTP Method
endpoint	TEXT	API URL

Purpose

Stores extracted API information.

Generated_Scripts Table

Column	Type	Description
id	INTEGER	Primary Key
user_id	INTEGER	Owner User
script_name	TEXT	Generated File Name
script_content	TEXT	Pytest Script

Purpose

Stores generated automation scripts.

AI_Recommendations Table

Column	Type	Description
id	INTEGER	Primary Key
recommendation	TEXT	Generated Recommendation
recommendation_type	TEXT	Positive/Negative/Security

Purpose

Stores generated testing recommendations.

9. AI INTEGRATION

The project uses a Hybrid Conversion Engine.

Rule-Based Engine

Handles standard assertions.

Example:

Postman:

```
pm.response.to.have.status(200);
```

Pytest:

```
assert response.status_code == 200
```

Gemini AI Engine

Handles:

- Complex assertions
- Conditional logic
- Advanced JavaScript validation

Advantages

- Faster conversion
 - Better accuracy
 - Reduced manual effort
-

10. SAMPLE INPUT AND OUTPUT

Sample Postman Assertion

```
pm.response.to.have.status(200);  
pm.expect(pm.response.json().id).to.eql(1);
```

Generated Pytest Assertion

```
assert response.status_code == 200  
assert response.json()["id"] == 1
```

Assertion Mapping

Postman Assertion	Generated Pytest Assertion
pm.response.to.have.status(200)	assert response.status_code == 200
pm.expect(...).to.eql(...)	assert value == expected

11. SECURITY FEATURES

The application implements:

Password Hashing

Passwords are hashed using Werkzeug before storage.

Session Authentication

Authenticated users are managed using Flask sessions.

Protected Routes

Only authenticated users can access dashboards and reports.

User Isolation

All records are linked using `user_id` to ensure one user cannot access another user's data.

12. DEPLOYMENT

Deployment Architecture

GitHub ↓ Render ↓ Flask Application ↓ SQLite Database

Deployment Platform

Render Cloud Platform

Features

- Automatic deployment
 - Cloud hosting
 - Online accessibility
 - Secure environment variable management
-

13. CONCLUSION

The Postman Collection to Pytest Migrator successfully automates the migration of Postman API test collections into executable Pytest automation scripts. By combining Rule-Based Conversion and AI-Assisted Conversion, the platform reduces manual effort, improves testing efficiency, and accelerates automation adoption.

The project demonstrates the practical application of AI in software engineering while providing a scalable solution for API testing migration challenges. Future enhancements can include PostgreSQL support, batch processing, CI/CD integration, and enterprise-scale deployment.